Name: Simangaliso Mdluli

Student Number: 21000300

Lab3

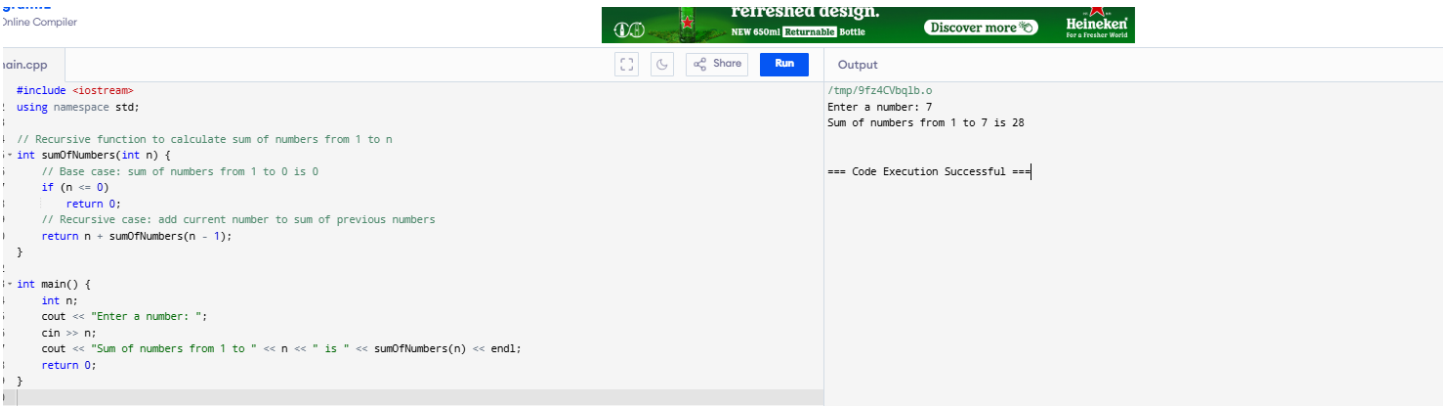
1.

Recursion and iteration both solve problems by repeatedly applying a process, but they differ in structure and resource usage. Recursion involves a function calling itself, leading to multiple function calls stored on the call stack, which increases memory usage and can cause stack overflow for deep recursion. Iteration uses loops (e.g., for or while) to repeat operations, with lower memory overhead as it doesn't require additional stack frames. While recursion can make code more elegant and easier to understand (e.g., tree traversal), iteration is generally more efficient in terms of performance for large datasets. Use recursion when the problem is naturally recursive (like tree structures or the Tower of Hanoi) and iteration when performance and memory optimization are critical.

2.

```
#include <iostream>

using namespace std;


// Recursive function to calculate sum of numbers from 1 to n
```

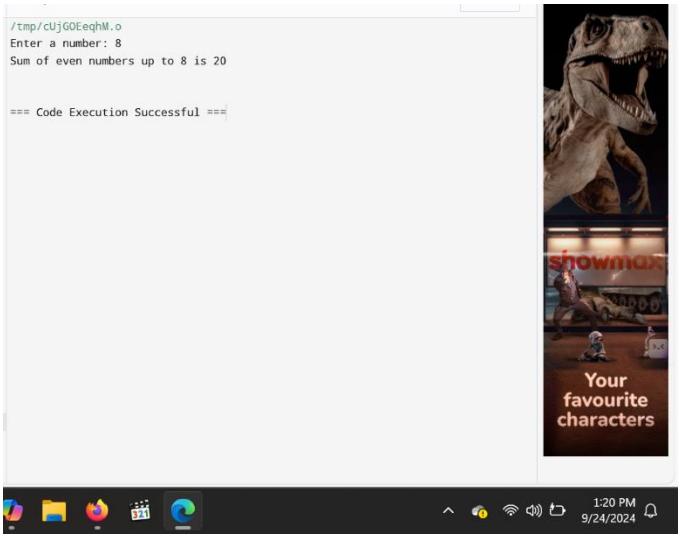
```
int sumOfNumbers(int n) {
   // Base case: sum of numbers from 1 to 0 is 0
   if (n <= 0)
      return 0;
   // Recursive case: add current number to sum of previous numbers
   return n + sumOfNumbers(n - 1);
}

int main() {
   int n;
   cout << "Enter a number: ";
   cin >> n;
   cout << "Sum of numbers from 1 to " << n << " is " << sumOfNumbers(n) << endl;
   return 0;
}
```



3.

```
#include <iostream>
using namespace std;
```

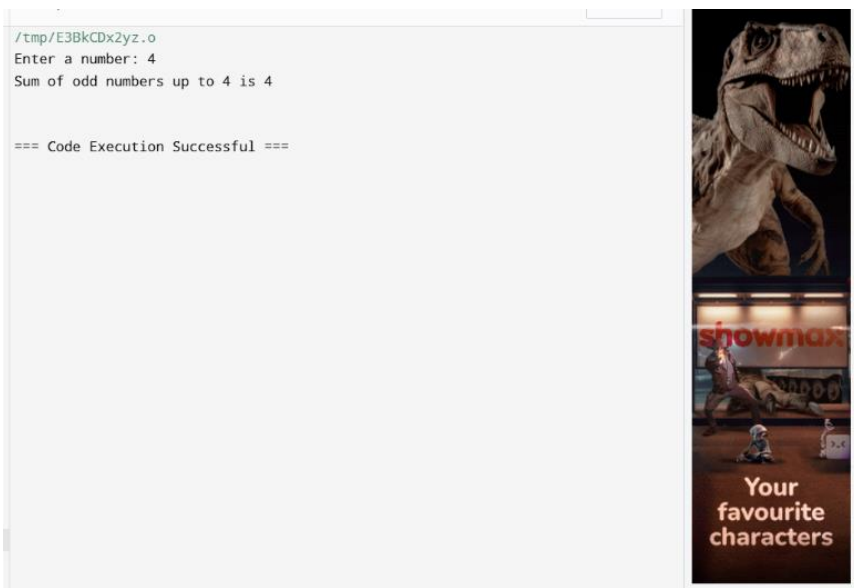
```
int sumOfEvenNumbers(int n) {

   if (n <= 0)
      return 0;

   if (n % 2 == 0)
      return n + sumOfEvenNumbers(n - 2);
   else
      return sumOfEvenNumbers(n - 1);
}

int main() {
   int n;
   cout << "Enter a number: ";
   cin >> n;
   cout << "Sum of even numbers up to " << n << " is " << sumOfEvenNumbers(n) << endl;
   return 0;
}
```



4.

```
#include <iostream>

using namespace std;


int sumOfOddNumbers(int n) {

  // Base case: sum of odd numbers up to 0 is 0

  if (n <= 0)

    return 0;


  if (n % 2 != 0)

    return n + sumOfOddNumbers(n - 2);

  else

    return sumOfOddNumbers(n - 1);

}


int main() {
```

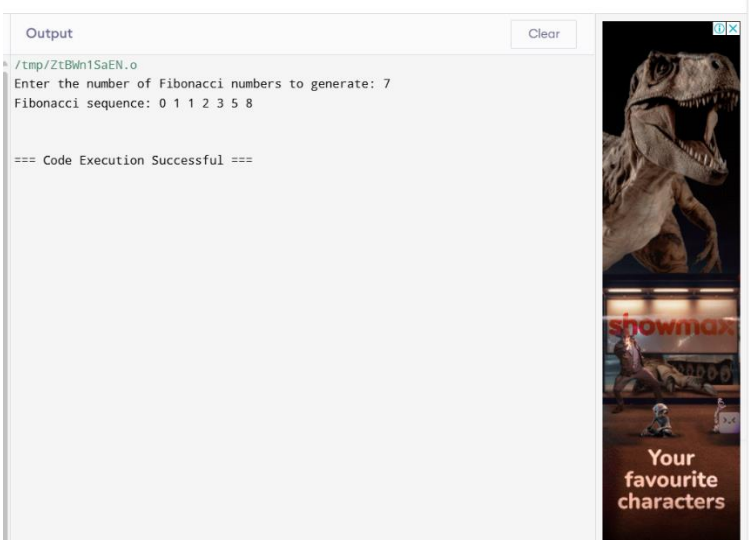
```
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Sum of odd numbers up to " << n << " is " << sumOfOddNumbers(n) << endl;
    return 0;
}
```



5.

```
#include <iostream>
using namespace std;

// Recursive function to generate the nth Fibonacci number
int fibonacci(int n) {
    // Base cases: F(0) = 0, F(1) = 1
```

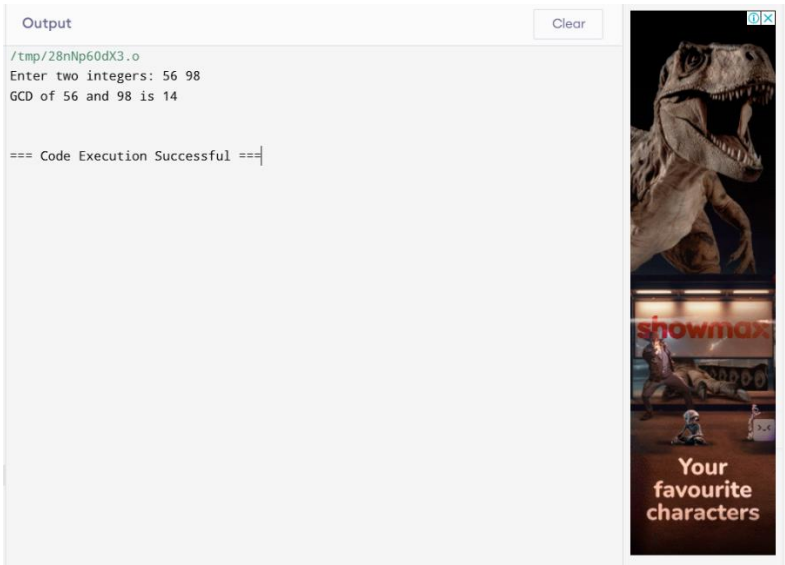
```
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    // Recursive case: F(n) = F(n-1) + F(n-2)
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n;
    cout << "Enter the number of Fibonacci numbers to generate: ";
    cin >> n;

    cout << "Fibonacci sequence: ";
    for (int i = 0; i < n; i++) {
        cout << fibonacci(i) << " ";
    }
    cout << endl;
    return 0;
}
```



6.

```
#include <iostream>

using namespace std;


// Recursive function to compute GCD using Euclidean algorithm

int gcd(int a, int b) {

   // Base case: if b is 0, GCD is a

   if (b == 0)

      return a;

   // Recursive case: GCD(a, b) = GCD(b, a % b)

   return gcd(b, a % b);

}


int main() {

   int a, b;

   cout << "Enter two integers: ";
```

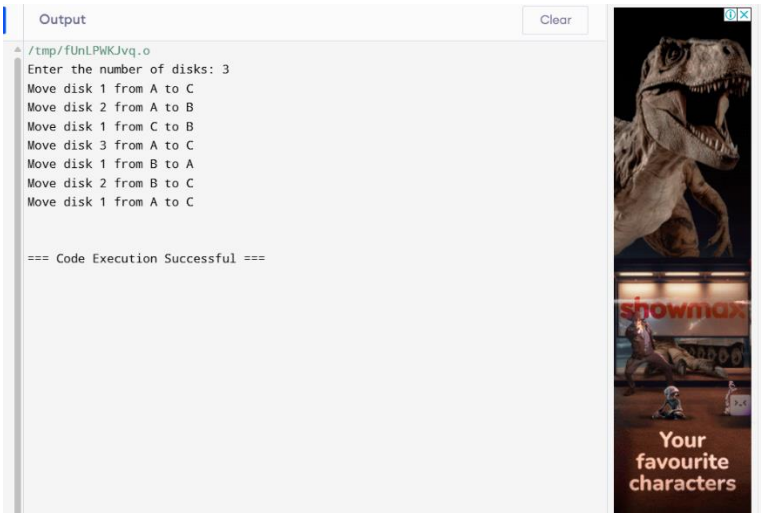
```
    cin >> a >> b;

    cout << "GCD of " << a << " and " << b << " is " << gcd(a, b) << endl;
    return 0;
}
```



7.

```
#include <iostream>
using namespace std;

// Recursive function to solve Tower of Hanoi puzzle
void towerOfHanoi(int n, char from_peg, char to_peg, char aux_peg) {
    if (n == 1) {
        cout << "Move disk 1 from " << from_peg << " to " << to_peg << endl;
        return;
```

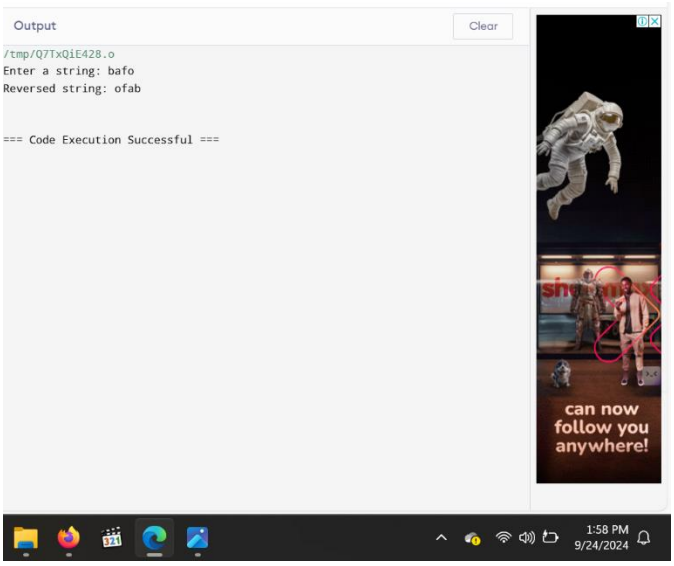
```
    }
    // Move n-1 disks from from_peg to aux_peg
    towerOfHanoi(n - 1, from_peg, aux_peg, to_peg);
    // Move nth disk from from_peg to to_peg
    cout << "Move disk " << n << " from " << from_peg << " to " << to_peg << endl;
    // Move n-1 disks from aux_peg to to_peg
    towerOfHanoi(n - 1, aux_peg, to_peg, from_peg);
}

int main() {
    int n;
    cout << "Enter the number of disks: ";
    cin >> n;

    towerOfHanoi(n, 'A', 'C', 'B');  // A = source, B = auxiliary, C = destination
    return 0;
}
```



8.

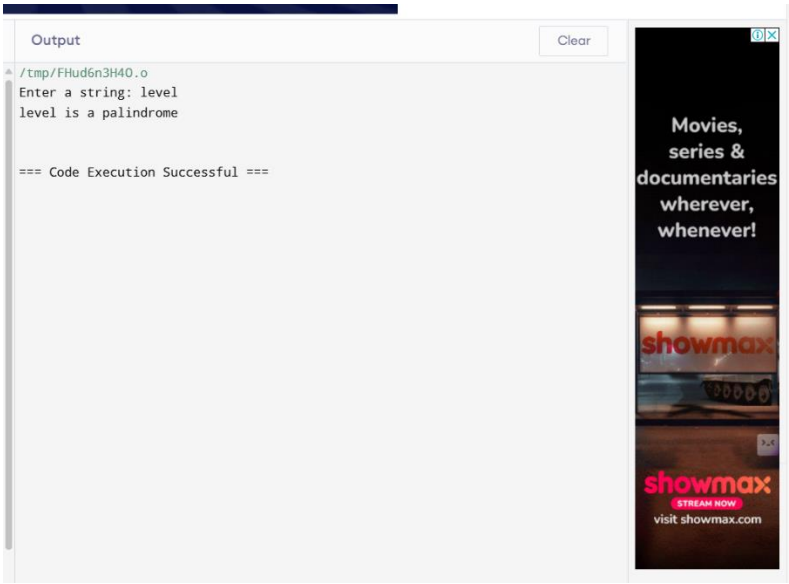
```
#include <iostream>
using namespace std;

// Recursive function to reverse a string
void reverseString(string &str, int start, int end) {
   // Base case: when start crosses end
   if (start >= end)
      return;
   // Swap the characters at start and end
   swap(str[start], str[end]);
   // Recurse for the remaining string
   reverseString(str, start + 1, end - 1);
}

int main() {
   string str;
```

```
    cout << "Enter a string: ";
    cin >> str;

    reverseString(str, 0, str.length() - 1);
    cout << "Reversed string: " << str << endl;
    return 0;
}
```




```
#include <iostream>
using namespace std;

// Recursive function to check if a string is a palindrome
bool isPalindrome(string &str, int start, int end) {
    // Base case: if start crosses end
    if (start >= end)
```

```
        return true;
    // If characters at start and end are not the same, it's not a palindrome
    if (str[start] != str[end])
        return false;
    // Recurse for the rest of the string
    return isPalindrome(str, start + 1, end - 1);
}

int main() {
    string str;
    cout << "Enter a string: ";
    cin >> str;

    if (isPalindrome(str, 0, str.length() - 1))
        cout << str << " is a palindrome" << endl;
    else
        cout << str << " is not a palindrome" << endl;

    return 0;
}
```



10.

```
#include <iostream>
using namespace std;

// Recursive function to convert decimal to binary
void decimalToBinary(int n) {
   // Base case: when the number becomes 0
   if (n == 0)
      return;
   // Recursively call the function with n / 2
   decimalToBinary(n / 2);
   // Print the remainder (n % 2)
   cout << n % 2;
}
```

```
int main() {
    int n;
    cout << "Enter a decimal number: ";
    cin >> n;

    if (n == 0) {
        cout << "Binary: 0" << endl;
    } else {
        cout << "Binary: ";
        decimalToBinary(n);
        cout << endl;
    }

    return 0;
}
```

Output

Clear

```
/tmp/9hWTVd8VVe.o
Enter a decimal number: 10
Binary: 1010


=== Code Execution Successful ===
```